Chapter 2 — Runtime Model

Field	Value
Status	Architect draft complete; constitutional and engineering review complete
Version	0.1
Scope	Boot, lifecycle, supervision, operating modes, degradation, shutdown, and recovery
Constitutional basis	Decisions 17–25, 28–29, 35, 38–45, 47, 50–51, 54, and 56
Owner review	None required at this stage

1. Purpose

The runtime model defines how HAL exists while operating. It specifies the order in which HAL earns the right to become active, the state transitions it may enter, how it detects and contains failure, and how it recovers without changing its constitutional rules. A running process is not HAL merely because it starts; it becomes an active HAL runtime only after identity, policy, protected state, and health requirements have been verified.

2. Runtime Principles

- Bootstrap constitutional controls before optional intelligence or execution services.
- Treat every runtime state as explicit, observable, and auditable.
- Supervise services independently; isolate fault domains rather than letting one failure cascade.
- Permit capability loss and safe degradation, but never silent alteration of governance, authority, identity, trust, authentication, or policy.
- Prefer recovery, replay, and verification over unexamined restart.
- Make resource use, health, uncertainty, and operating limits part of the runtime self-model.

3. Runtime Constituents

Constituent	Runtime responsibility	Failure consequence
Bootstrap Manager	Builds the verified runtime in dependency order and selects the initial operating mode.	HAL remains unavailable or enters Safe Mode.
Constitutional Kernel	Establishes root identity, policy and authorization authority, protected audit, and commit coordination.	No constitutional mutation or protected action may proceed.
Service Supervisor	Starts, observes, isolates, restarts, and retires governed services.	Affected capability is degraded or suspended.
Resource Governor	Accounts for CPU, memory, GPU,	Admission control or bounded

Constituent	Runtime responsibility	Failure consequence
	storage, network, power, thermal, and attention budgets.	degradation applies.
Mode Manager	Maintains the authoritative operating mode and allowed transitions.	Defaults to the more restrictive safe state.
Health and Evidence Collector	Collects liveness, readiness, integrity, performance, and cognitive-health evidence.	Confidence falls; unverified components are restricted.
Recovery Manager	Coordinates journals, checkpoints, replay, restoration, and gradual rejoin.	Recovery holds the affected domain until verification completes.

4. Startup and Admission Sequence

Startup is a staged admission process. Each stage establishes evidence required by the next. A stage may halt or restrict the runtime; no stage may be skipped because a dependent service is unavailable.

Stage	Required evidence	Outcome
0. Hardware and host admission	Host identity, secure-time availability or bounded uncertainty, storage accessibility, minimum resource baseline.	Host is accepted as a candidate runtime host.
1. Kernel bootstrap	Root identity material, constitutional version, protected configuration, audit journal integrity.	Constitutional Kernel becomes available in restricted bootstrap state.
2. State admission	Ledger consistency, checkpoint validation, policy version verification, secrets reference availability.	Authoritative state is opened read-only until recovery checks pass.
3. Core-service admission	Identity, policy, audit, transaction, mode, supervisor, and observability readiness.	Core control plane becomes ready.
4. Capability admission	Node/provider identities, capability manifests, health, current delegation, treaty status.	Capabilities are registered but not necessarily schedulable.
5. Active admission	Readiness quorum, current Self Model, verified runtime mode, declared limitations.	HAL accepts normal or degraded work consistent with mode.

5. Runtime State Machine

The Mode Manager owns the current runtime mode. Every transition creates an immutable mode-transition event containing cause, evidence, affected capabilities, authority, and recovery criteria.

Mode	Permitted behavior	Exit criteria
Bootstrap	Identity and protected-state verification only. No external actions.	Kernel and state admission succeeds.
Normal	All healthy, authorized capabilities may operate under ordinary policy and transaction controls.	Detected degradation, scheduled maintenance, emergency, or shutdown.
Degraded	Healthy capabilities operate with disclosed limitations; scheduler excludes unfit components.	Required health and verification evidence restore confidence.
Restricted	Local reasoning, monitoring, evidence collection, and safe read operations; no protected permanent mutations.	Partition or integrity condition is reconciled and verified.
Safe Mode	Read-only inspection, recovery, audit, and Owner-authorized repair paths only.	Root cause is addressed and recovery verification passes.
Recovering	Journal replay, reconciliation, restore, integrity checks, and staged re-admission.	Authoritative state and readiness gates pass.
Maintenance	Approved scoped maintenance; unrelated work is isolated or deferred.	Maintenance change is verified, rolled back, or concluded.
Emergency	Priority routing for an active emergency under explicit emergency policy.	Emergency closes; post-incident review and reconciliation begin.
Offline	No external connectivity assumed. Local rules still apply; network-dependent capabilities are unavailable.	Connectivity and peer identity are re-established and verified.

6. Service Lifecycle and Supervision

Every governed service declares identity, dependency contracts, readiness criteria, resource limits, failure behavior, restart policy, state ownership, and evidence produced. The supervisor may restart a failed service, move work, rebuild a projection, or isolate a node. It may not modify the service's policy, delegation, constitutional role, or persistent authoritative data outside the established recovery path.

- A service transitions through Defined, Starting, Ready, Busy, Draining, Stopped, Failed, Quarantined, or Recovering.
- Liveness proves that a service is reachable; readiness proves it is fit to receive a declared workload. Neither substitutes for authorization or trust.
- Restart budgets prevent a crash loop from consuming resources or creating noisy, misleading health evidence.
- Stateful services drain or checkpoint before planned retirement. Stateless services may be replaced after contract compatibility and provenance checks.
- A quarantined service or node may collect evidence but receives no work that can change protected state until re-admitted.

7. Resource and Attention Admission

The Resource Governor evaluates physical and cognitive resource budgets before work begins. It uses the scheduling and attention systems to select the smallest adequate execution scope. Resource exhaustion is a source of evidence, not an excuse to bypass controls.

Budget	Examples of enforcement
Physical	CPU, GPU, memory, storage, thermal, power, and network headroom; admission limits and node fitness checks.
Cognitive	Reasoning, planning, retrieval, simulation, reflection, and learning allocations; attention budgets and starvation review.
Safety	Maximum concurrency for physical actions, transaction blast radius, rollback capacity, and verification capacity.
Privacy and treaty	Data classification, permitted destinations, retention, and disclosure bounds.
Time	Deadlines, time uncertainty, lease duration, retry horizon, and stale-evidence limits.

8. Failure Detection and Containment

Failure detection combines heartbeat, readiness, integrity, behavior, resource, and cross-subsystem evidence. The system avoids interpreting a missing heartbeat as a definitive explanation. It records a hypothesis, confidence, scope, and recommended containment action.

Failure class	Containment response
Single optional service	Isolate, restart within budget, route to a compatible alternative, and disclose degraded capability.
Stateful core service	Suspend affected mutations; preserve evidence; fail closed or safe according to the service declaration.
Node or provider integrity concern	Quarantine execution authority, collect evidence, require controlled recovery before rejoin.

Failure class	Containment response
Network partition	Enter Restricted mode for isolated partitions; prohibit canonical commits and governance actions until reconciliation.
Resource exhaustion	Apply admission control, defer low-priority work, reduce optional workloads, and preserve critical recovery capacity.
Constitutional integrity concern	Enter Safe Mode; stop protected mutations; escalate to Owner Authorization Ceremony for any repair requiring protected change.

9. Shutdown, Handoff, and Recovery

A planned shutdown is a transaction: HAL drains active work, records handoff state, closes or compensates transactions, checkpoints durable state, preserves audit and mode evidence, and verifies that recovery material is available. An unplanned shutdown is treated as an incident. On restart, HAL does not assume previous work succeeded; it reconstructs state from durable facts and verification.

- Graceful shutdown stops new work first, then drains reversible work, then handles protected transactions according to their recovery contract.
- Every in-flight transaction has a durable status: pending, committed, compensated, abandoned with evidence, or awaiting Owner decision.
- Recovery replays authoritative journals, verifies checkpoints, identifies uncertain effects, and uses reconciliation rather than duplicate execution.
- Nodes and services rejoin gradually: identity, integrity, policy, trust, state synchronization, health, and capability fitness are separately verified.
- Restoration is not complete until a recovery verification record proves the restored system can read, reason, enforce policy, and execute only within its permitted mode.

10. Runtime Observability and Self Model

The runtime publishes evidence to the operational and cognitive self-model. It reports current mode, active limitations, health dimensions, resource headroom, dependency status, admission decisions, restart history, recovery progress, time confidence, and calibration concerns. A status response is evidence-backed; a language model may render it, but it does not invent it.

11. Runtime Guarantees

- HAL MUST establish constitutional identity and protected state before accepting protected work.
- HAL MUST record mode changes, recovery transitions, and authoritative state changes as durable evidence.
- HAL MUST NOT treat local network location, a successful heartbeat, or service availability as sufficient trust or authorization.
- HAL MUST NOT silently change its rules as a consequence of degradation, partition, overload, recovery, or emergency.
- HAL MUST apply the declared failure behavior of each capability and make reduced capability visible to affected participants.
- HAL MUST preserve an auditable path from request, through runtime mode and resource admission, to execution evidence and outcome.

12. Constitutional Traceability Audit

Constitutional decisions	Chapter 2 implementation coverage
17–25	Logical execution nodes, resource limits, lifecycle, work scheduling, capability allocation, recovery, secure communication, consistency, policy enforcement.
28–29	Operational/cognitive self-awareness, health forecasting inputs, fault tolerance, split-brain restrictions, self-healing boundaries, failure declaration.
35, 38–45	Transactions, supervision, configuration integrity, observability, resources, durable recovery, safe lifecycle change, temporal confidence, Presence continuity.
47, 50–51	Distributed coordination, reality modes, verification gates, self-description, identity continuity, constitutional mirror inputs.
54, 56	Attention budgets, starvation prevention inputs, explicit limits, uncertainty-aware restriction and escalation.

13. Internal Engineering Review

Check	Result
Constitutional conflicts	None identified. Runtime behavior preserves Book I authority boundaries.
Lifecycle completeness	Startup, admission, active operation, degradation, containment, shutdown, and recovery are covered.
Failure posture	Mode transitions, service isolation, resource admission, and partition restrictions are explicit.
Unnecessary coupling	Specific process managers, databases, and orchestration frameworks are intentionally deferred.
Owner review required	None. The chapter defines implementation mechanics without changing constitutional philosophy, values, or authority.

14. Completion Status

Chapter 2 is complete. It establishes the runtime contract that later service, event, storage, and deployment chapters must satisfy. Chapter 3 will define the Constitutional Kernel and the narrow enforcement interfaces on which the runtime depends.